

CS525 Final Project Report

LAMBDA to JavaScript Compiler

Spring 2026

1. Church's Lambda-Calculus

Lambda calculus is a formal system that Alonzo Church came up with in the 1930s. The whole idea is that you can express any computation using just three things: variables (like x), abstractions (like $\lambda x.M$, which is basically an anonymous function), and applications (like $M N$, which applies M to N).

Computation works through beta-reduction. When you apply $\lambda x.M$ to some argument N , you just substitute N for x everywhere in M . So $(\lambda x. x + x)(3)$ reduces to $3 + 3 = 6$.

What's cool is that even though it's super minimal, you can encode everything with it: numbers (Church numerals), booleans, pairs, recursion (Y combinator), all of it. I actually did this for assignment 5 where I compiled the extended lambda calculus down to pure LC using Church encodings. The LAMBDA language we're compiling here is basically an extended version of this with actual types and data structures.

2. The LAMBDA Language

LAMBDA is the source language for this compiler. Programs are represented as ASTs using two datatypes: `styp` for types and `dexp` for expressions.

2.1 Types

`STbas(name)` is for base types: `int`, `bool`, `string`, `unit`. Pretty self-explanatory.

`STxyz` is an existential type variable. It has a mutable ref cell that starts as `None`. During type inference, unification fills it in. This is how we do Hindley-Milner style inference: you create fresh type variables and unify them as you walk the AST.

`STtup(T1, T2)` is a pair type, `STfun(T1, T2)` is a function type $T1 \rightarrow T2$.

`STlazy(T)` is for lazy values (thunks), `STlist(T)` for lists, `STarray(T)` for arrays, `STstcn(T)` for streams.

2.2 Expressions

`DEint`, `DEbtf`, `DEstr` are literals. They type to `int`, `bool`, and `string` respectively.

`DEvar(x)` is a variable reference, and its type comes from looking up x in the typing context.

`DElam(x, body)` is an untyped lambda. Since there's no annotation, we create a fresh type variable for x and infer the body to get the result type, giving us `STfun( $\alpha$ , body_type)`. `DElam1(x, T, body)` is the typed version where the argument type T is

given, so we just use that directly. `DEapp(f, a)` is function application, so `f` must have type `STfun(T1, T2)` where `T1` matches `a`'s type, and the whole thing types to `T2`.

`Defix(f, x, body)` is how you do recursion. It defines a function named `f` with parameter `x`, and `f` is in scope inside `body` so it can call itself. We create fresh variables for the arg and result types, bind `f` to `STfun( $\alpha$ ,  $\beta$ )` and `x` to α in the context, infer the `body`, and unify the `body`'s type with β . The whole thing types to `STfun( $\alpha$ ,  $\beta$ )`. `Defix1` is the same but with explicit type annotations so no fresh variables are needed.

`DEopr(name, args)` handles all the built-in operators. Arithmetic ops (`+`, `-`, `*`, `/`, `%`) take two ints and return int. Comparisons (`<`, `>`, `<=`, `>=`, `=`, `!=`) take two ints and return bool. Boolean ops (`&&`, `||`) take two bools and return bool. `not` takes a bool and returns bool. String operations like `string_append` take two strings and return string. Data structure ops are polymorphic, so `list_head` takes a `list( $\alpha$ )` and returns α , `array_get` takes `array( $\alpha$ )` and an int and returns α , `lazy_force` takes `lazy( $\alpha$ )` and returns α .

`DEtup(a, b)` makes a pair and types to `STtup(T1, T2)` where `T1` and `T2` are the types of `a` and `b`. `DEfst` and `DESnd` project out the components. They unify the argument with `STtup( $\alpha$ ,  $\beta$ )` and return α or β .

`DEif0(test, then, else)` is if-then-else. The test must have type bool, and both branches must have the same type, which is the result type.

`DElet(x, def, scope)` is let-binding. We infer the type of `def`, bind `x` to that type in the context, and infer `scope`.

`DElist_nil` types to `list( $\alpha$ )` with a fresh variable. `DElist_cons(hd, tl)` unifies `tl` with `list(hd_type)` and returns that list type.

`DElazy(e)` suspends evaluation and types to `lazy(T)` where `T` is `e`'s type.

`DEstcn_nil` types to `stcn( $\alpha$ )` with a fresh variable. `DEstcn_cons(hd, tl)` requires the tail to be `lazy(stcn(hd_type))` and types to `stcn(hd_type)`.

`DEarray_size_val(n, v)` requires `n` to be int and types to `array(v_type)`.

`DEarray_size_fun(n, f)` requires `n` to be int and `f` to be `int  $\rightarrow$   $\alpha$` , and types to `array( $\alpha$ )`.

`DEanno(e, T)` is a type annotation. It infers `e`'s type and unifies it with `T`, so it acts as a constraint.

3. Type-Checking

Type inference is done by `dexp_tinfer(de0)` which calls `dexp_tinfer01(de0, ctx)` with an empty context. The context is just a linked list of (variable name, type) pairs that I extend as I go deeper into the AST.

The core mechanism is unification. I have a function `styp_unify(t1, t2)` that takes two types and tries to make them equal. It first normalizes both types by following any `STxyz` reference chains (`styp_norm`). If one of them is an unresolved `STxyz`, it gets bound to the other type. If both are constructed types like `STfun`, it unifies their

components recursively. If they don't match (like trying to unify `int` with `bool`), it throws an error. There's also an occurs check so you don't end up with infinite types.

For each expression form, the inference is pretty straightforward. Literals return their base type. For `DElam(x, body)`, I create a fresh type variable for `x`, add it to the context, infer the body type, and return `STfun(arg_type, body_type)`. For `DEapp(f, a)`, I infer `f` and `a`'s types, create a fresh result variable, and unify `f`'s type with `STfun(a_type, result)`. For `DEFix(f, x, body)`, I make fresh variables for the arg and result, put both `f` (with the function type) and `x` (with the arg type) in the context, infer the body, and unify.

Operators are handled by a helper `_opr_tinfer` that just maps each operator name to its expected types. Like `+` takes two ints and returns `int`, `=` takes two ints and returns `bool`, `list_head` takes a `list( $\alpha$ )` and returns `$\alpha$` , etc.

4. JS Code Emitter

The emitter is `dexp_trx2js(de0)`. It creates a `JSEmitter` object (basically a string buffer) and recursively walks the AST, appending JS code as it goes.

The translation is pretty direct:

Literals just emit themselves. `DEint(42)  $\rightarrow$  42`, `DEbtf(true)  $\rightarrow$  true`, strings get escaped and quoted.

Lambdas become JS function expressions: `DElam(x, body)  $\rightarrow$  (function(x) { return <body>; })`.

Application is just a function call: `DEapp(f, a)  $\rightarrow$  <f>(<a>)`.

Fix uses named function expressions for recursion: `DEFix(f, x, body)  $\rightarrow$  (function f(x) { return <body>; })`. Since the function has a name, it can call itself.

Let-binding uses an IIFE: `DElet(x, d, s)  $\rightarrow$  (function() { let x = <d>; return <s>; })()`. I use `let` instead of `var` so nested lets don't have scoping issues.

Tuples are JS arrays: `[a, b]`. `Fst` and `Snd` are `[0]` and `[1]`. If-then-else is a ternary. Lists use `[h].concat(t)`. Lazy values are thunks `(function() { return e; })`. Streams are `[head, tail]` pairs with null for empty. Arrays use `Array.fill` and `Array.from`.

Operators map to their JS equivalents: arithmetic is infix, `=` becomes `===`, `print` becomes `console.log`, etc.

Everything compiles to a single JS expression (no statements), which makes composition easy.

5. Closure Conversion

I didn't implement closure conversion. The JS output has nested functions that capture free variables through JavaScript's built-in closure mechanism. If I were to do it, I'd need

a pass that finds the free variables of each lambda/fix, replaces them with closure objects (a function pointer + an environment record), and lifts all functions to the top level. Application would then unpack the closure and pass the environment as an extra argument.

6. Testing

I tested with 7 programs that cover all the language constructs:

test_basics: int, bool, string literals, arithmetic, comparison.

test_lambda: typed and untyped lambda, application, let, if-then-else.

test_tuples: tuple construction, fst/snd, nested tuples.

test_recursion: factorial with DEfix and DEfix1, both naive and tail-recursive.

test_lists: nil, cons, head, tail, length, recursive sum.

test_advanced: arrays, lazy, streams, type annotations, Church numerals, higher-order map.

test_queens: full 8-queens backtracking solver. Finds all 92 solutions.

All 7 pass. Type inference works and the generated JS runs correctly in Node. You can run them with `bash TEST/run_all.sh`.